

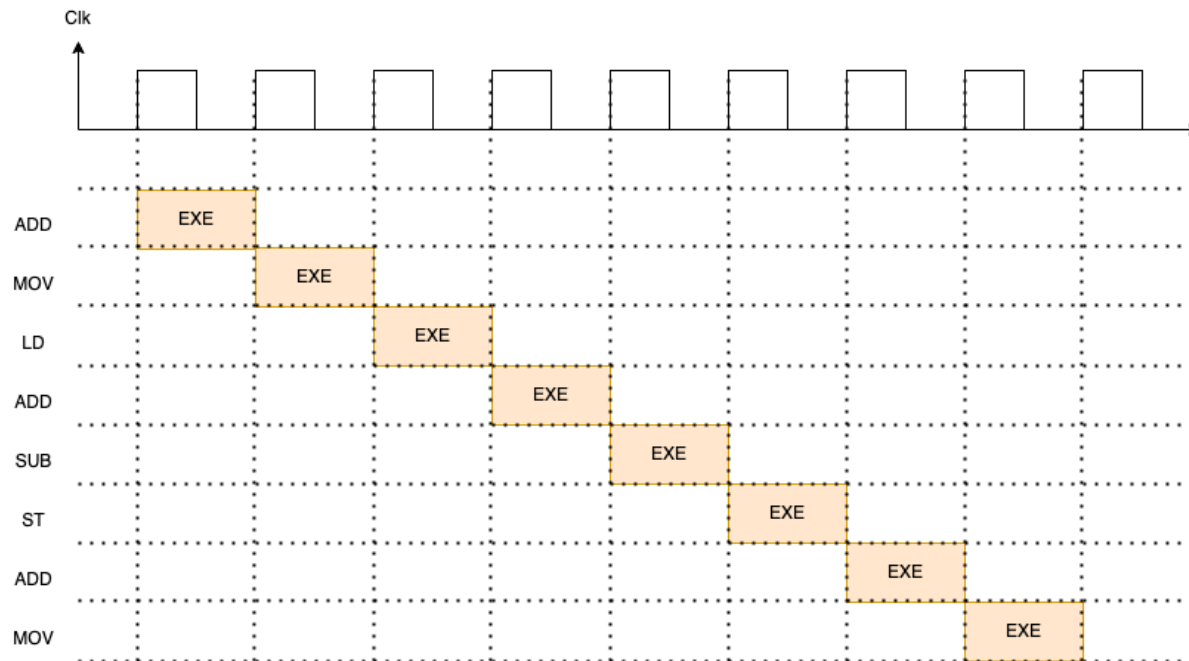
# Architecture des Processeurs

Micro-architecture d'un processeur Multi-Cycle

yann.douze@sorbonne-universite.fr

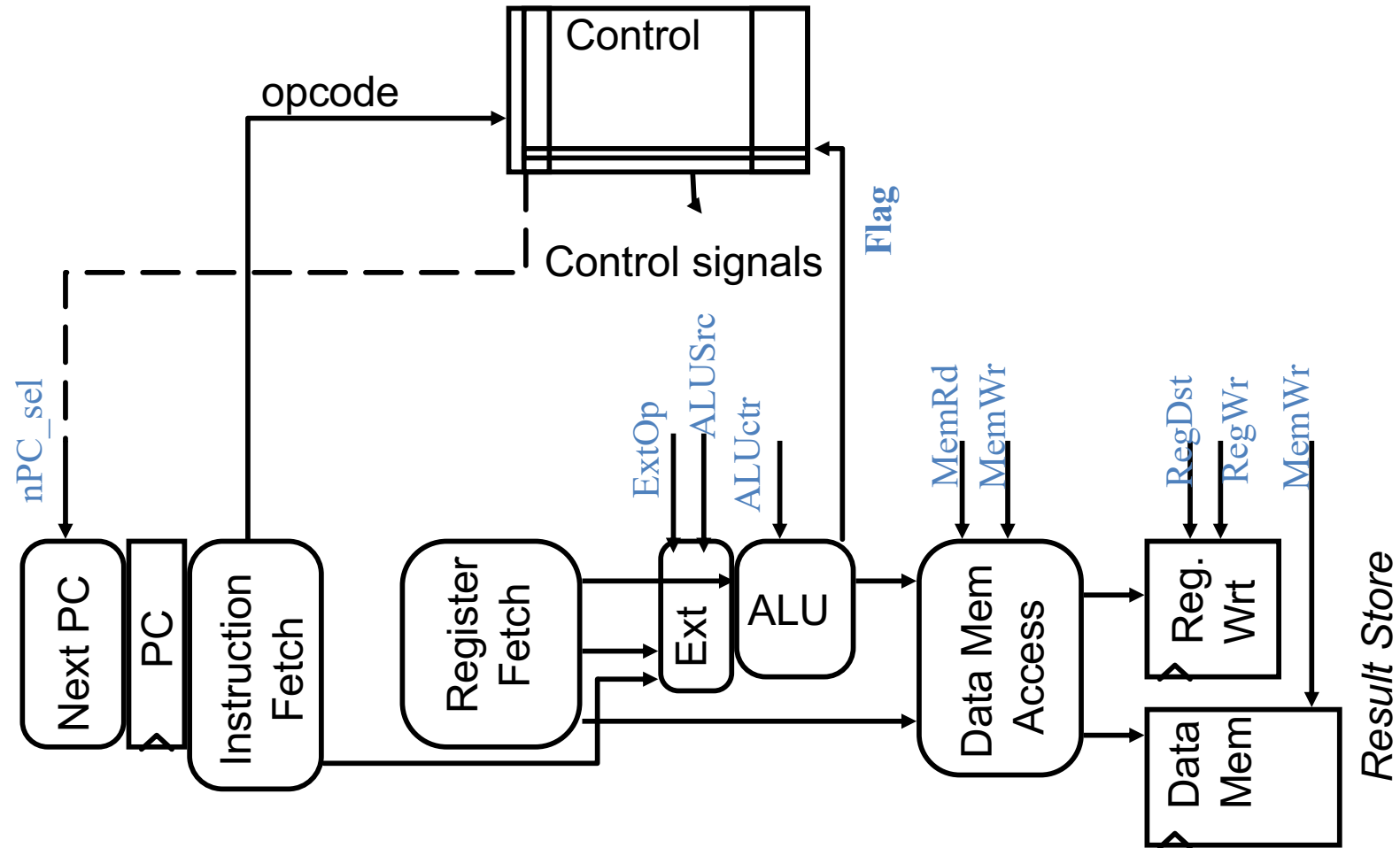
# Processeur monocycle

- Chaque instruction s'exécute en un seul cycle machine
- Nombre de cycle par instruction :  $CPI = 1$

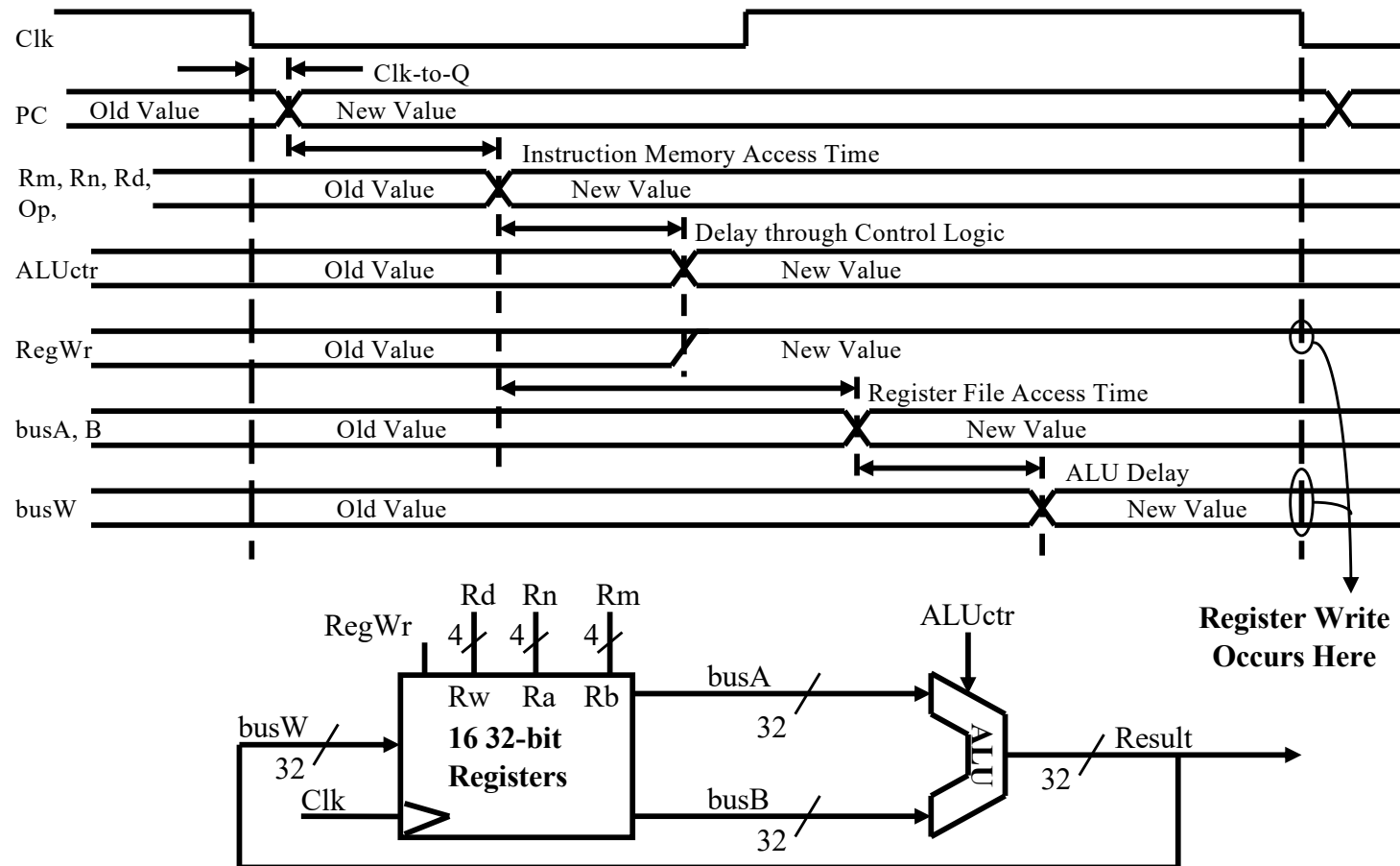


- **Avantage :**  
Chaque instruction s'exécute en 1 cycle machine
- **Inconvénient :**  
Temps de propagation dans la logique combinatoire élevé.  
Donc Période élevée et  $F_{max}$  est limité.

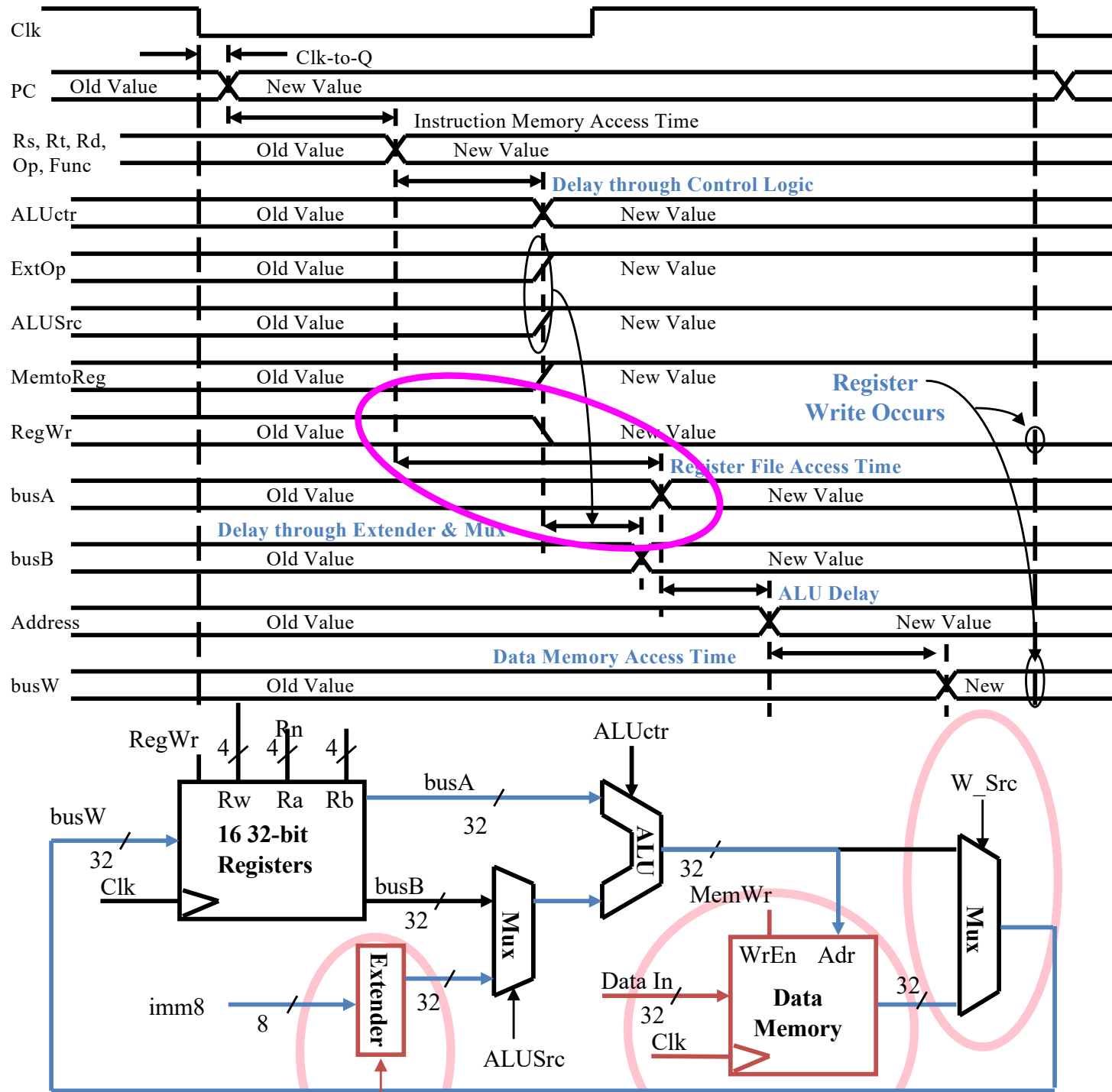
# Vue abstraite du processeur monocycle



## Timing de registre à registre pour l'instruction ADD Rd, Rn, Rm



# Chronogramme de l'instruction de chargement (Load)



# Quel est le problème avec notre processeur CPI = 1 ?

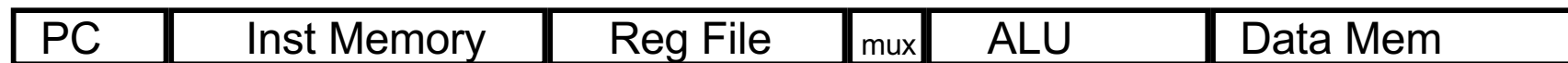
Arithmetic & Logical



Load



Store



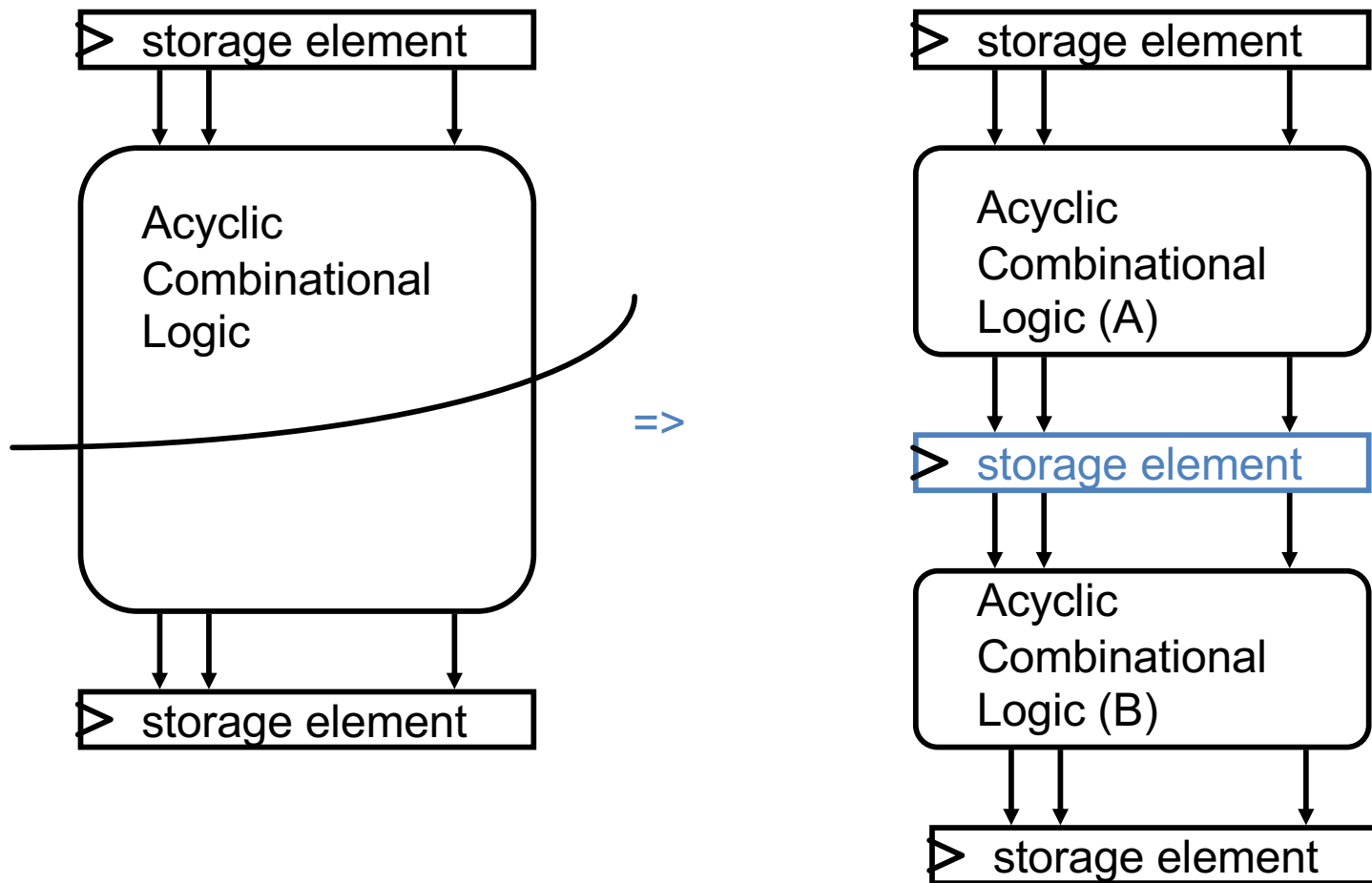
Branch



- Temps de cycle long
- Toutes les instructions prennent autant de temps que la plus lente

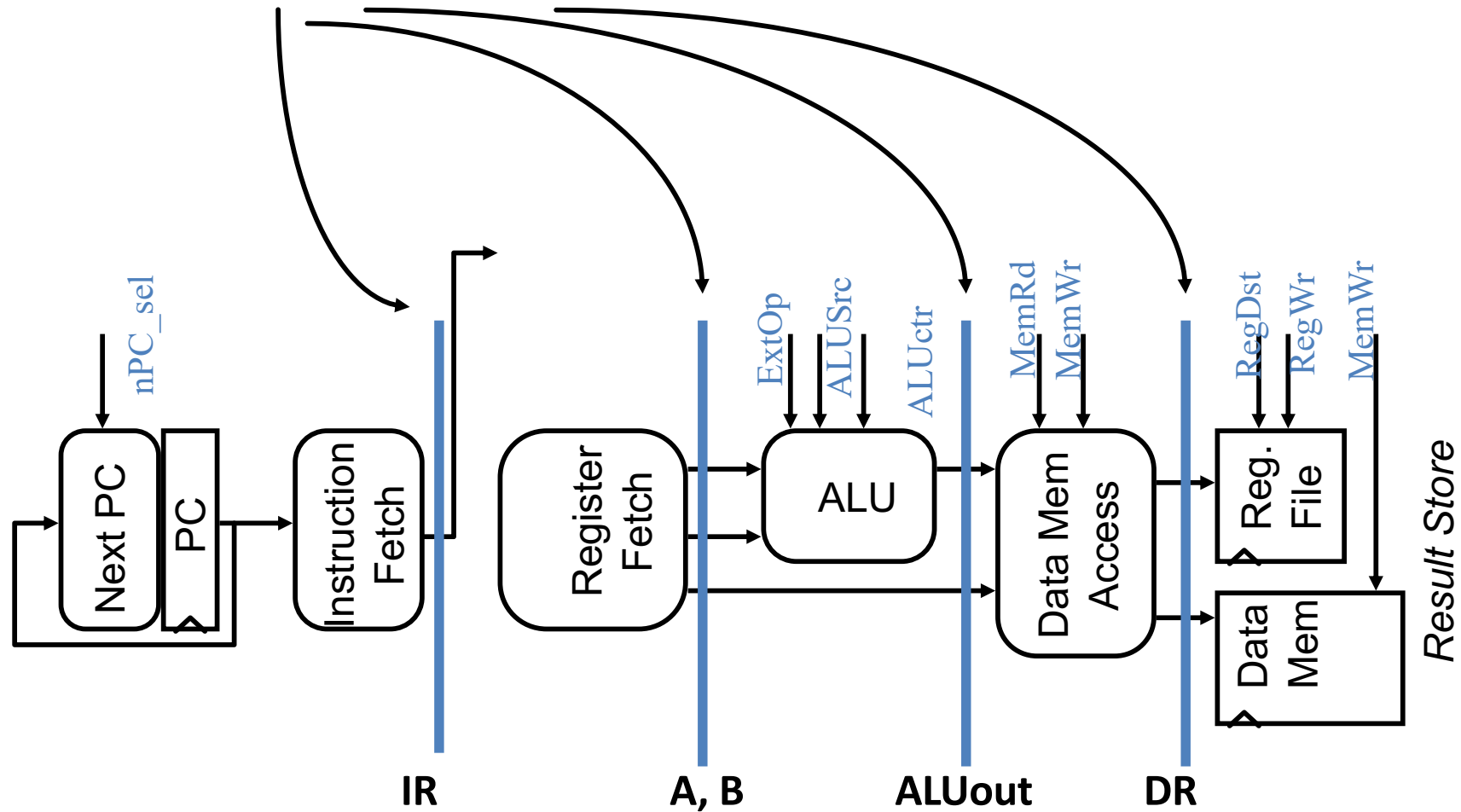
# Réduire le temps de cycle

- Couper le graphe de dépendances combinatoire et insérer des registres
- Effectuer le même travail en deux cycles rapides, plutôt qu'un lent



# Partitionnement du chemin de données

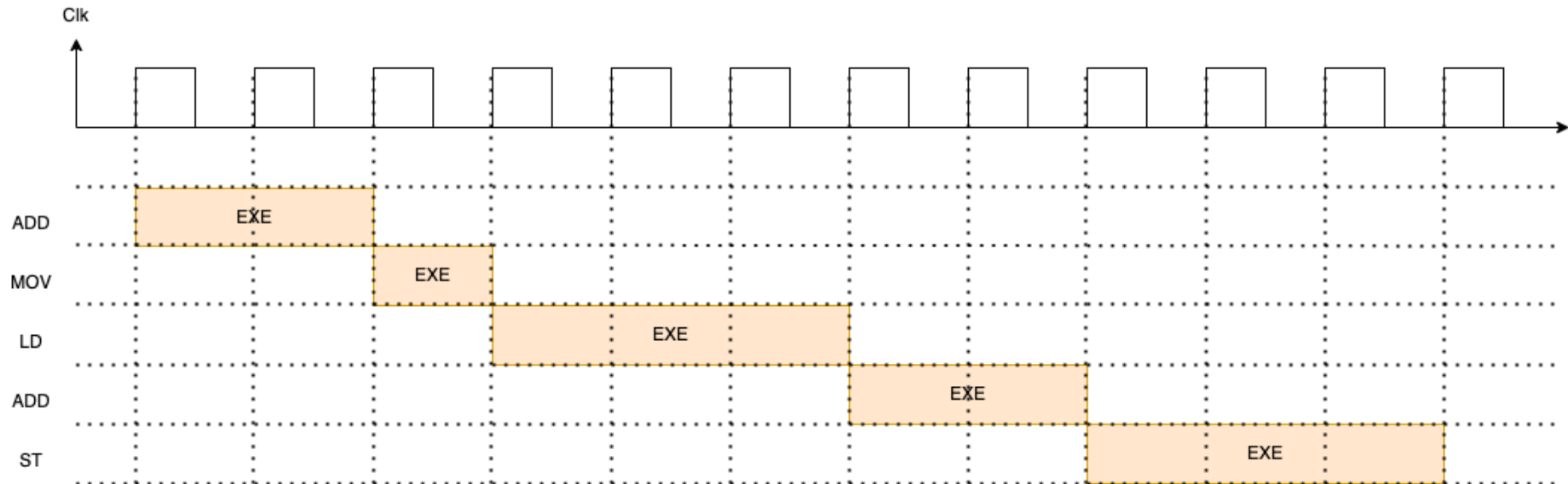
- Ajouter des registres entre chaque étape



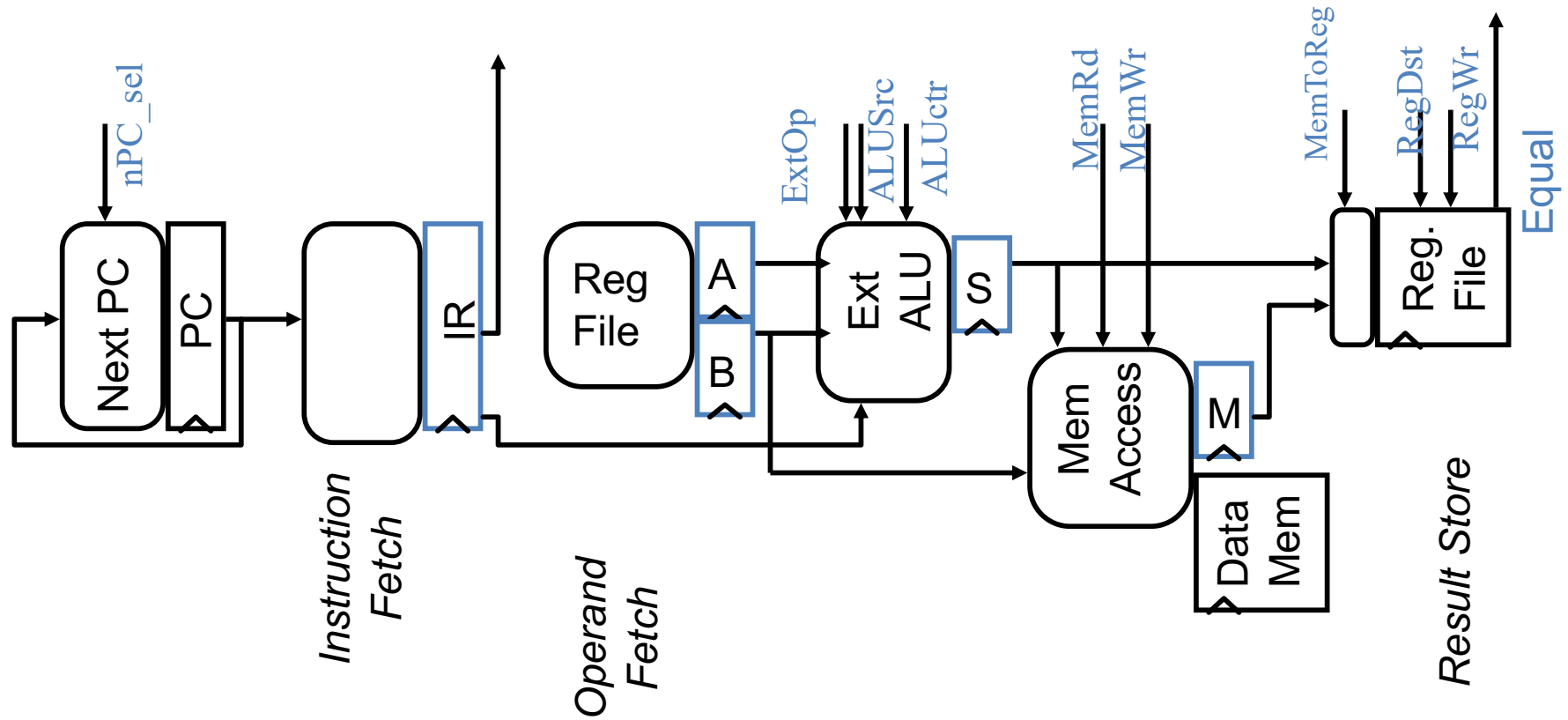


# Processeur multicycle

- Chaque instruction s'exécute en plusieurs cycles machines qui peut être variable.
- Nombre de cycle par instruction : CPI  $\gg 1$
- Mais on peut augmenter la fréquence car les blocs combinatoires sont plus petits



# Chemin de données Multicycle Multicycle DataPath



# Invoke step-by-step processor design technique

Etape 1: ISA => Analyser le jeu d'instructions

Etape 2: Composants du chemin de données

Etape 3: Assembler les composants afin de réaliser le chemin de données

Etape 4 : Etablir la logique de controle du processeur monocycle

Etape 5: Chemin de données + transferts de registres Logique  
=> Transferts de registres physique

Les transferts de registres logique utilisent les registres de l'ISA

Les transferts de registres physiques vont utiliser des registres intermédiaires en plus

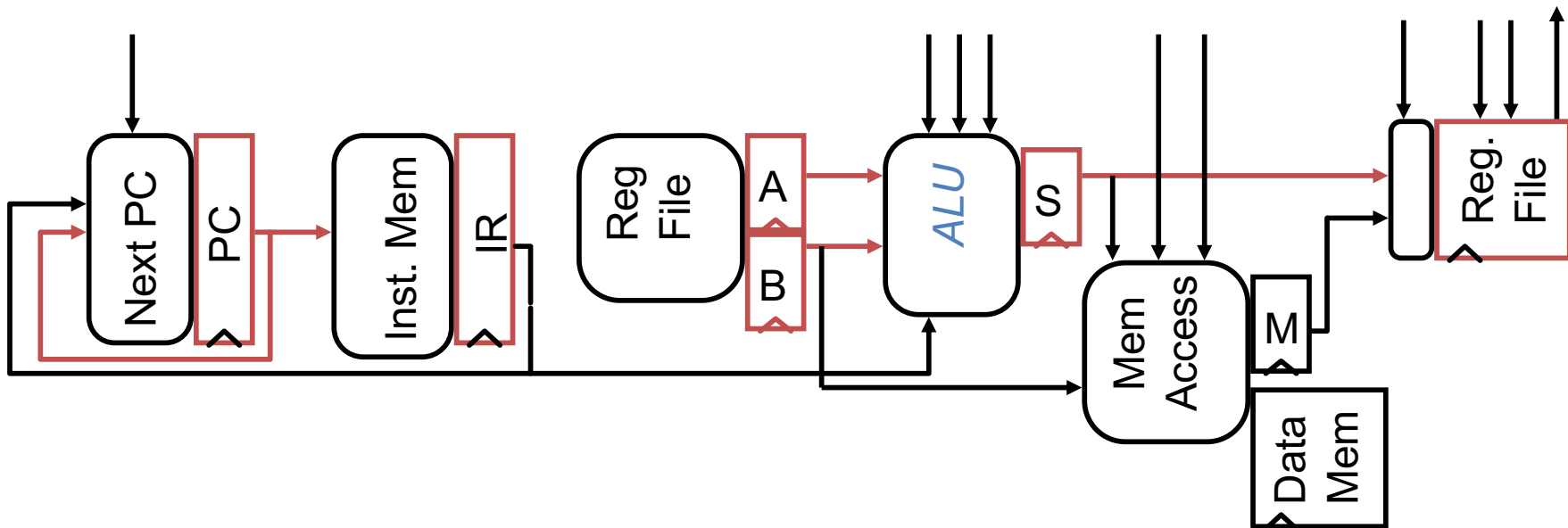
Etape 6 : Controle des transferts de registres physique

# Etape 5: operation arithmétique entre registre

- Logical Register Transfer
- Physical Register Transfers

| <u>inst</u> | <u>Logical Register Transfers</u>                      |
|-------------|--------------------------------------------------------|
| ADD         | $R[rd] \leftarrow R[rn] + R[rm]; PC \leftarrow PC + 1$ |

| <u>inst</u> | <u>Physical Register Transfers</u>               |
|-------------|--------------------------------------------------|
|             | $IR \leftarrow MEM[pc]$                          |
| ADD         | $A \leftarrow R[rn]; B \leftarrow R[rm]$         |
|             | $S \leftarrow A + B$                             |
|             | $R[rd] \leftarrow S; \quad PC \leftarrow PC + 1$ |



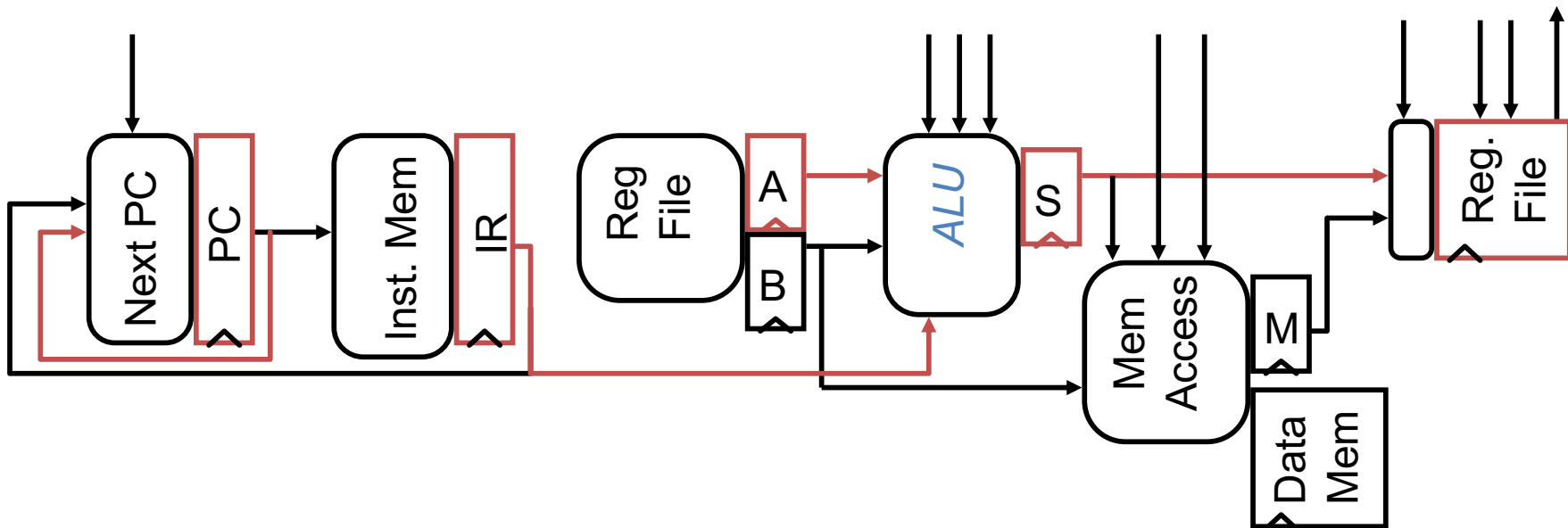
# Etape 5: operation arithmétique entre un registre et une valeur immediate

- Logical Register Transfer
- Physical Register Transfers

## inst      Logical Register Transfers

ADD       $R[rd] \leftarrow R[rn] \text{ ADD (imm8); } PC \leftarrow PC + 1$

| <u>inst</u> <u>Physical Register Transfers</u> |                                                                                           |
|------------------------------------------------|-------------------------------------------------------------------------------------------|
|                                                | $IR \leftarrow MEM[pc]$                                                                   |
| ADD                                            | $A \leftarrow R[rn], B \leftarrow R[rm]$ <span style="color: magenta;">side effect</span> |
|                                                | $S \leftarrow A + (imm8)$                                                                 |
|                                                | $R[rd] \leftarrow S; \quad PC \leftarrow PC + 1$                                          |



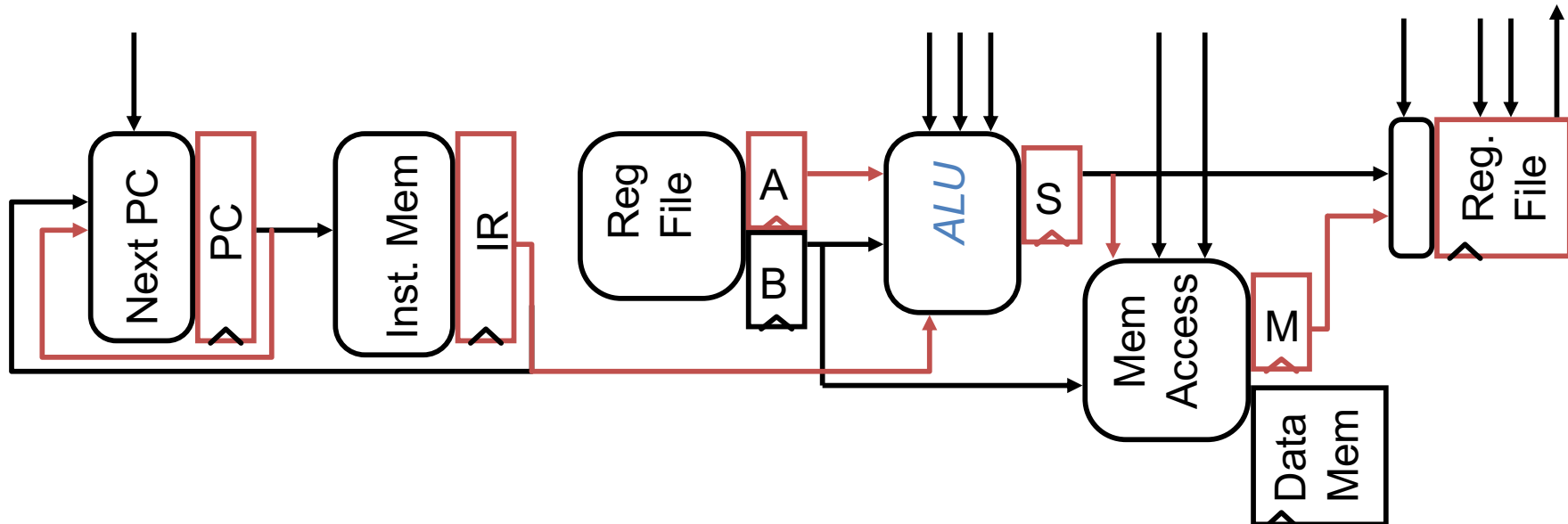
# Etape 5 : Load

- Logical Register Transfer
- Physical Register Transfers

inst      Logical Register Transfers

**LDR**       $R[rd] \leftarrow MEM(R[rn] + sx(imm8));$   
 $PC \leftarrow PC + 1$

| <u>inst</u> | <u>Physical Register Transfers</u>               |
|-------------|--------------------------------------------------|
|             | $IR \leftarrow MEM[pc]$                          |
| <b>LDR</b>  | $A \leftarrow R[rn]; B \leftarrow R[rt]$         |
|             | $S \leftarrow A + SignEx(imm8)$                  |
|             | $M \leftarrow MEM[S]$                            |
|             | $R[rd] \leftarrow M; \quad PC \leftarrow PC + 1$ |



# Etape 5 : Store

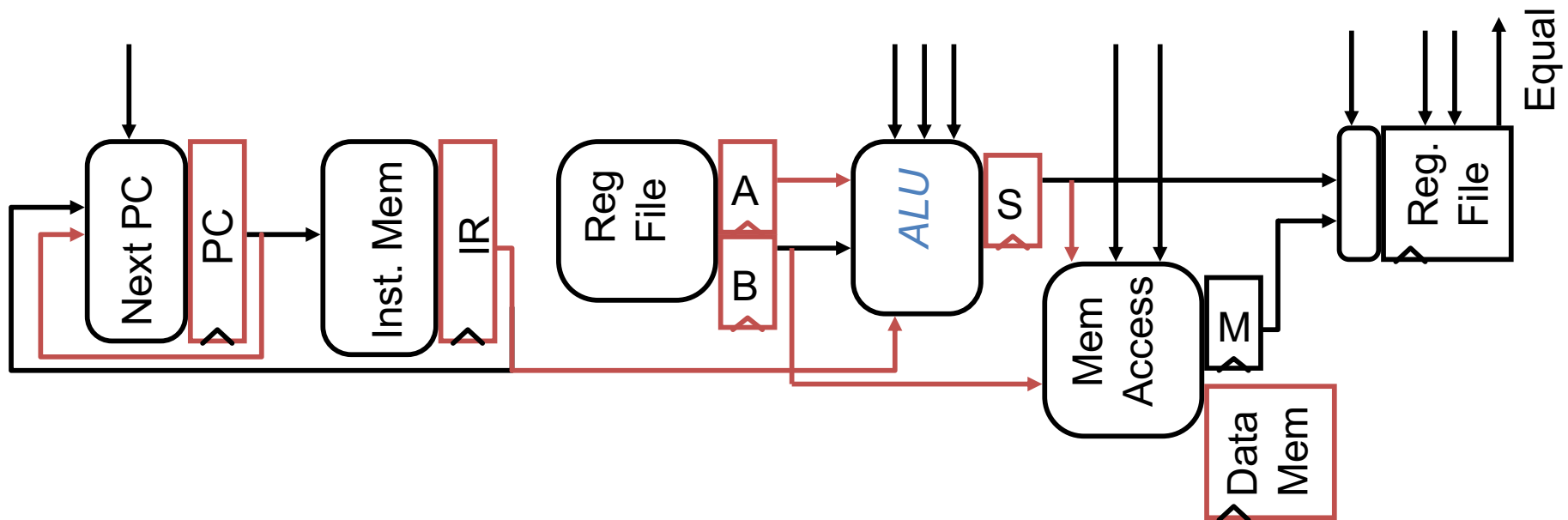
- Logical Register Transfer
- Physical Register Transfers

inst      Logical Register Transfers

STR       $\text{MEM}(\text{R}[\text{rn}] + \text{sx}(\text{imm8}) \leftarrow \text{R}[\text{rd}];$

$\text{PC} \leftarrow \text{PC} + 1$

| <u>inst</u> | <u>Physical Register Transfers</u>                                                  |
|-------------|-------------------------------------------------------------------------------------|
|             | $\text{IR} \leftarrow \text{MEM}[\text{pc}]$                                        |
| STR         | $\text{A} \leftarrow \text{R}[\text{rn}]; \text{B} \leftarrow \text{R}[\text{rd}]$  |
|             | $\text{S} \leftarrow \text{A} + \text{SignEx}(\text{imm8});$                        |
|             | $\text{MEM}[\text{S}] \leftarrow \text{B} \quad \text{PC} \leftarrow \text{PC} + 1$ |



# Etape 5: Branch

- Logical Register Transfer

inst      Logical Register Transfers

BLT      if flag N=1

          then  $PC \leq PC + sx(imm24)$

          else  $PC \leq PC + 1$

- Physical Register Transfers

inst      Physical Register Transfers

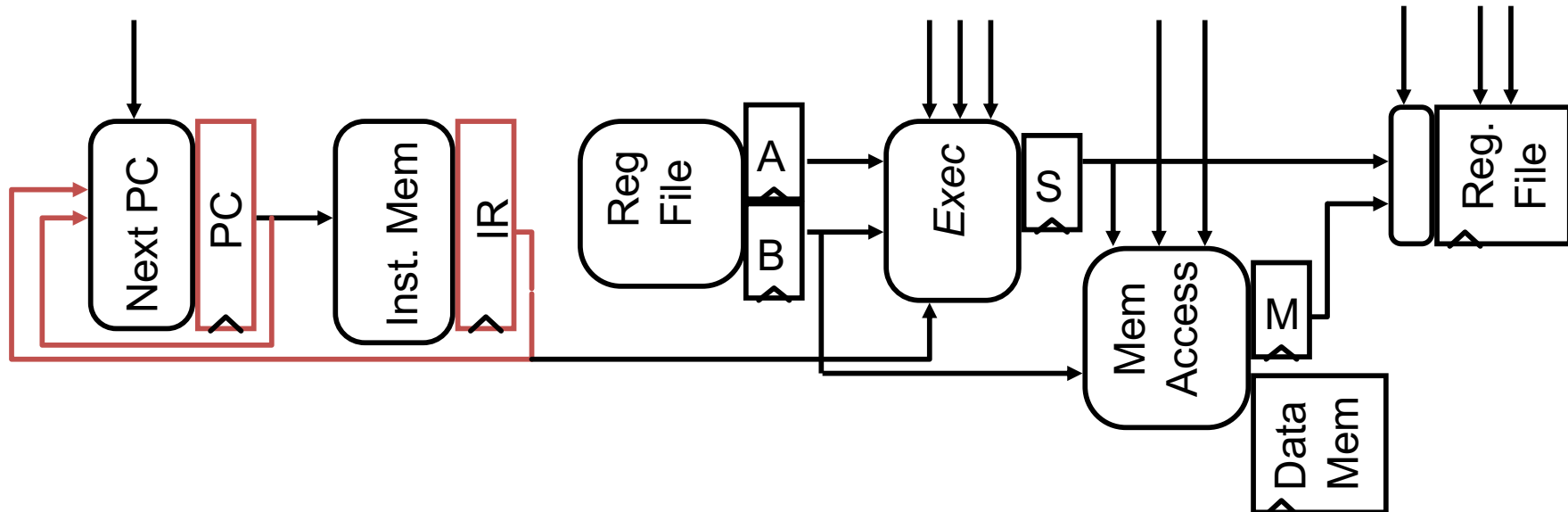
          IR  $\leftarrow$  MEM[pc]

BLT, N=0  $\rightarrow$       PC  $\leftarrow$  PC + 1

inst      Physical Register Transfers

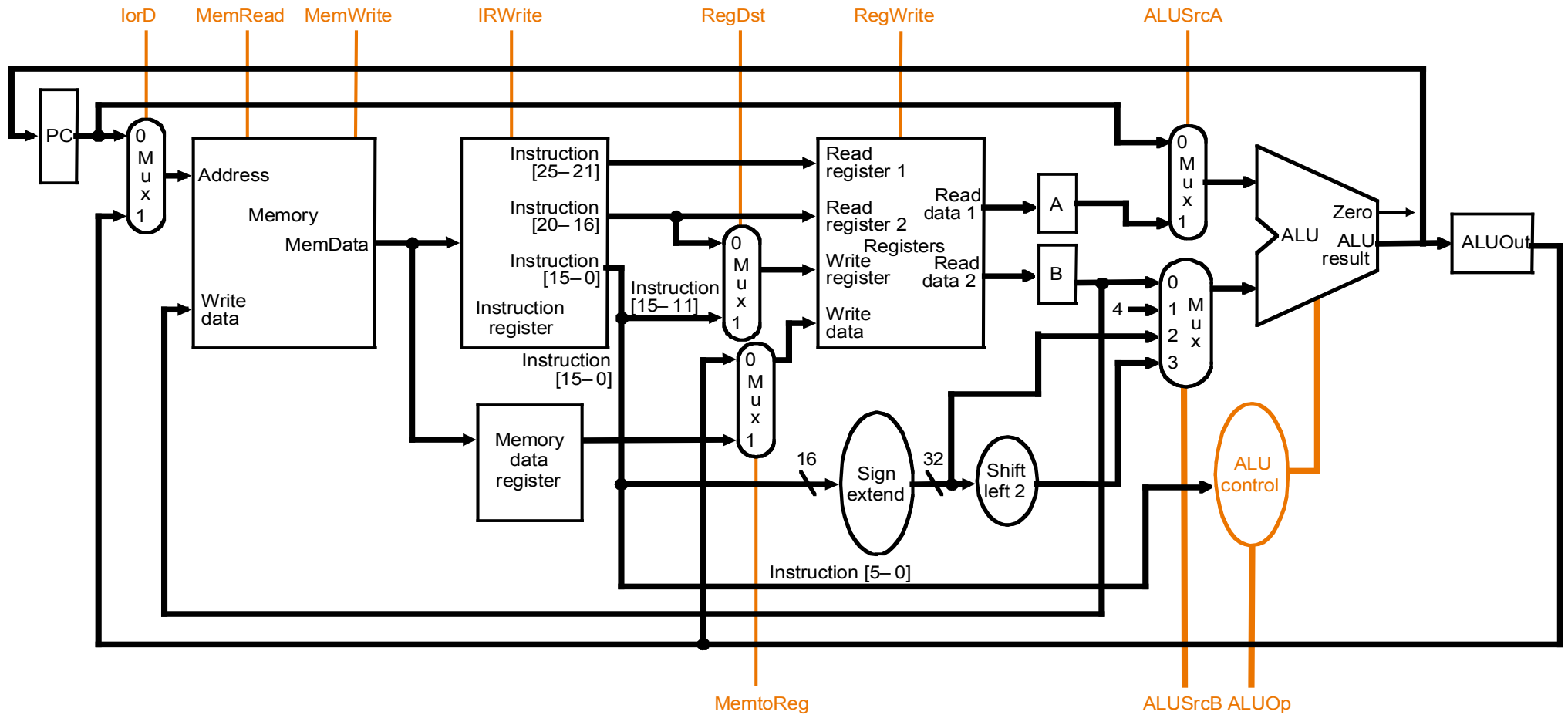
          IR  $\leftarrow$  MEM[pc]

BLT, N=1  $\rightarrow$       PC  $\leftarrow$  PC + sx(imm24)



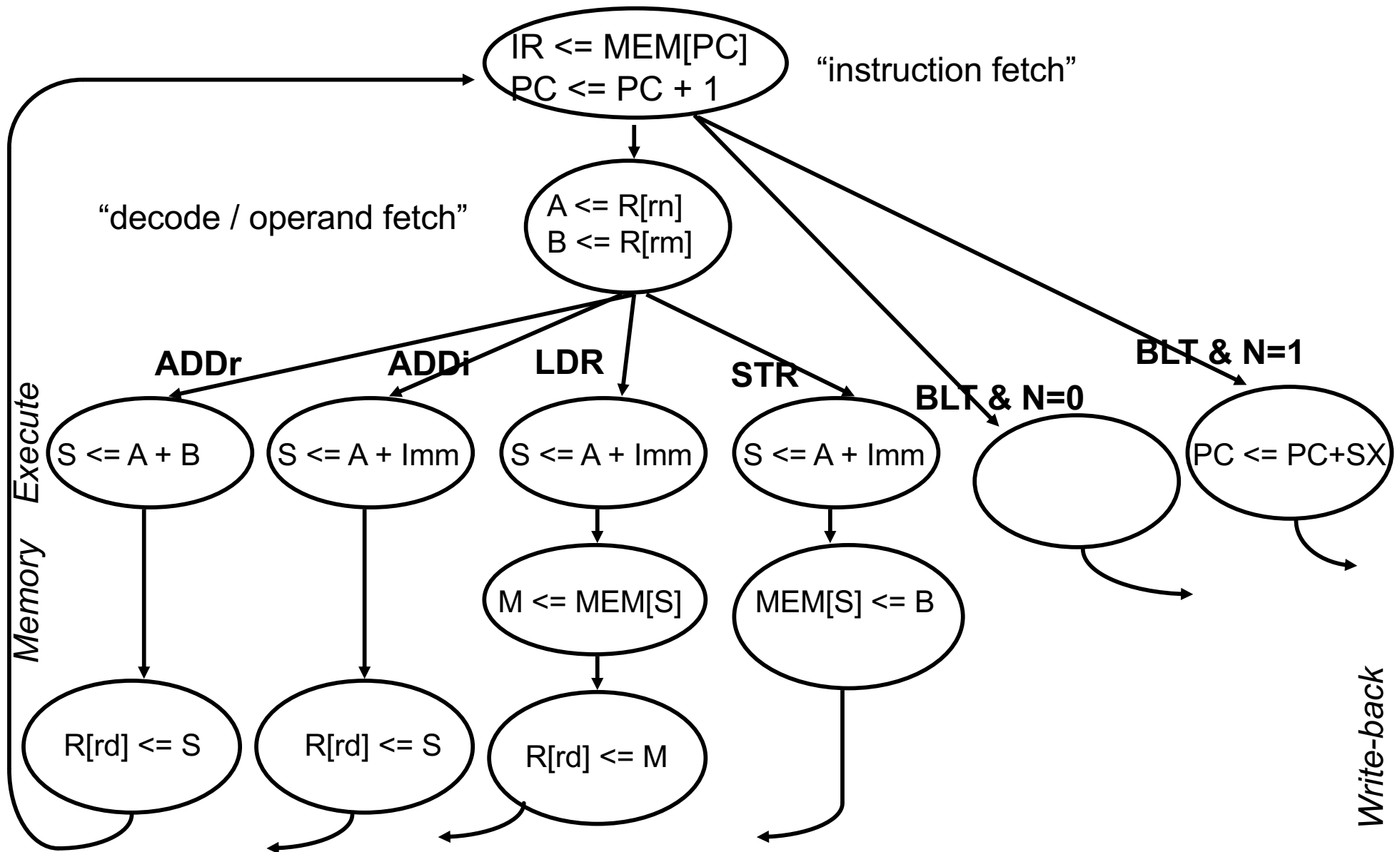


# Multiple cycle datapath with control lines



## Etape 6

### Logique de controle pour le chemin de données multicycle



# Processeur pipeliné : cycle d'instruction du ARM7TDMI

- **1. Phase de recherche ("fetch")**
  - Recherche de l'instruction en mémoire
- **2. Phase d'analyse ("decode")**
  - Décodage de l'instruction: analyse de la représentation binaire 32 bit de l'instruction et de ses opérandes pour mise en œuvre des signaux par l'unité de commande
  - Pour les phases suivantes, 2 cas possibles selon qu'il s'agit d'une instruction de traitement (sollicite l'unité de traitement) ou d'une instruction d'accès mémoire (sollicite les unités de traitement et mémoire)

Pour une instruction de traitement de données:

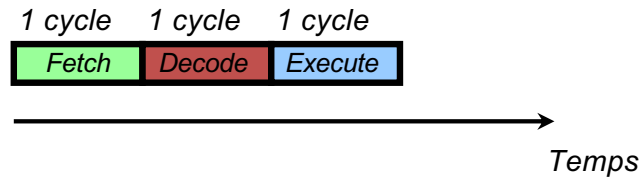
- **3. Phase d'exécution ("execute")**
  - Circulation des données, sélection des opérations, exécution conditionnelle

Pour une instruction d'accès mémoire:

- **3. Phase de calcul d'adresse ("address calc")**
  - L'adresse est calculée par l'UAL et stockée dans le registre d'adresse
- **4. Phase d'accès mémoire ("data transfer")**
  - L'adresse est envoyée à la mémoire qui lit/écrit la donnée

# Exécution pipeline

- Sur l'ARM7, les phases "fetch", "decode" et "execute" prennent chacune un cycle d'horloge
- Chaque instruction de traitement s'exécute donc en trois cycles d'horloge



Convention des couleurs:

- Vert: utilisation de la mémoire (fetch lit les instructions en mémoire)
- Rouge: utilisation du décodeur/séquenceur (analyse des instructions)
- Bleu: utilisation de l'unité de traitement (exécution d'une instruction de traitement)

# Exécution pipeline

- Combien de temps pour exécuter la séquence d'instructions suivante ?

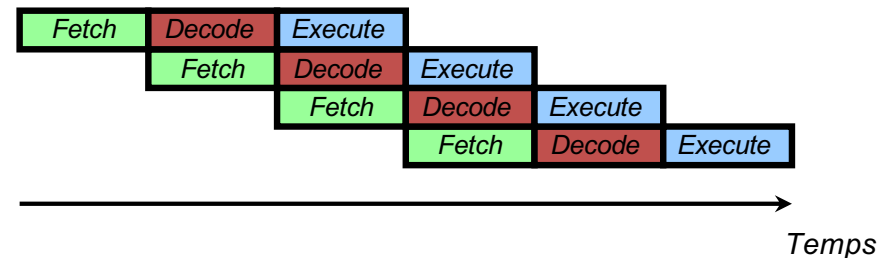
```
MOV r5, r3, LSL #1
ADD r5, r5, r3, LSL #3
MOV r7, r5, LSL #1
ADD r5, r5, r3, LSL #3
```

- 4 instructions x 3 = 12 cycles d'horloge ?

# Exécution pipeline

- Réponse: 6 cycles d'horloge  
!!

```
MOV r5, r3, LSL #1
ADD r5, r5, r3, LSL #3
MOV r7, r5, LSL #1
ADD r5, r5, r3, LSL #3
```

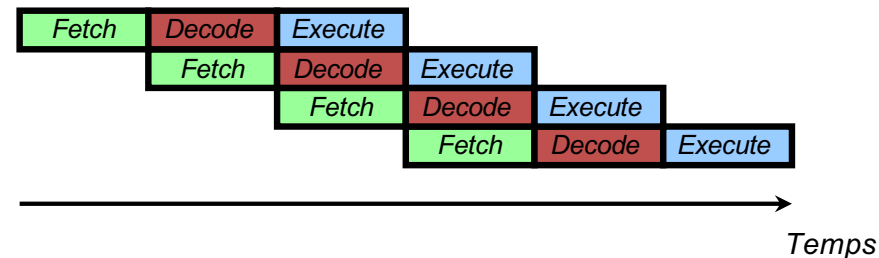


- Le pipeline permet de recouvrir les phases d'exécution
  - La division en phases indépendantes (fetch, decode, execute) permet de démarrer l'exécution d'une instruction avant que la précédente ne soit terminée
  - On peut démarrer la phase fetch dès la phase decode de l'instr précédente
  - On peut démarrer la phase decode dès la phase execute de l'instr précédente

# Exécution pipeline

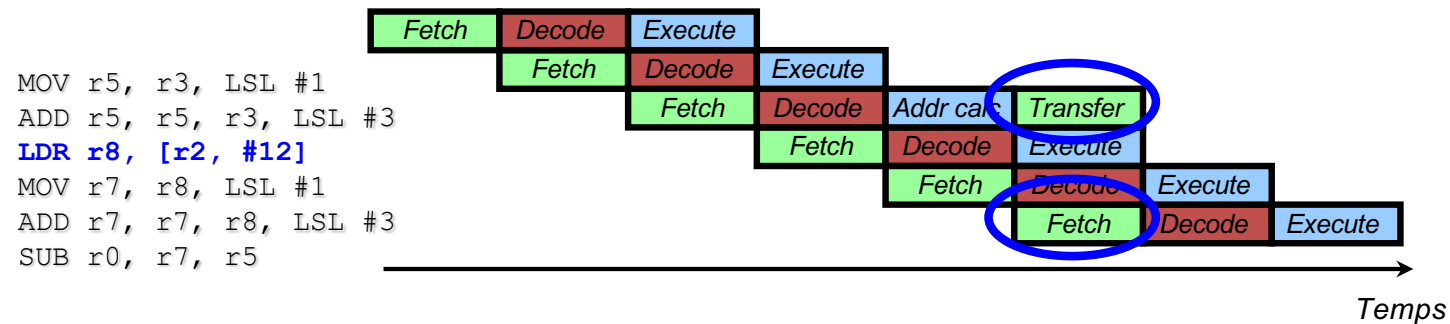
- Réponse: 6 cycles d'horloge !!

```
MOV r5, r3, LSL #1
ADD r5, r5, r3, LSL #3
MOV r7, r5, LSL #1
ADD r5, r5, r3, LSL #3
```



- On peut exécuter une instruction par cycle d'horloge
  - A partir du cycle 3, une phase d'exécution se termine à chaque cycle d'horloge
  - On utilise au mieux les ressources: 3 phases s'exécutent parfois simultanément
  - **Condition: ne pas avoir 2 fois la même couleur dans le même cycle!!**

# Ralentissement du pipeline: cas des instructions d'accès mémoire (1)

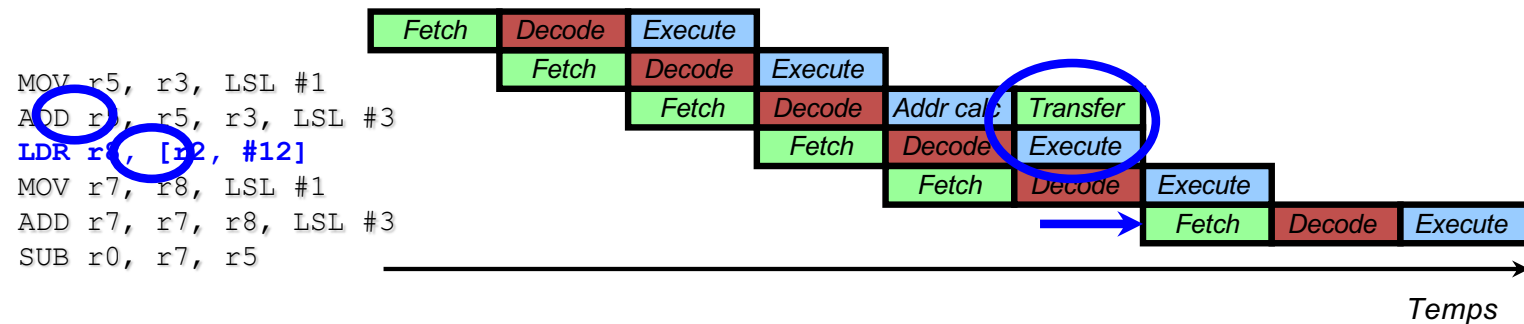


On insère une instruction d'accès mémoire (LDR r8, [r2, #12])

- La phase addr calc utilise l'unité de traitement (bleu)
- La phase de transfert utilise la mémoire (vert)
- Les deux étapes transfert (LDR) et fetch (SUB) ne peuvent pas utiliser la mémoire simultanément



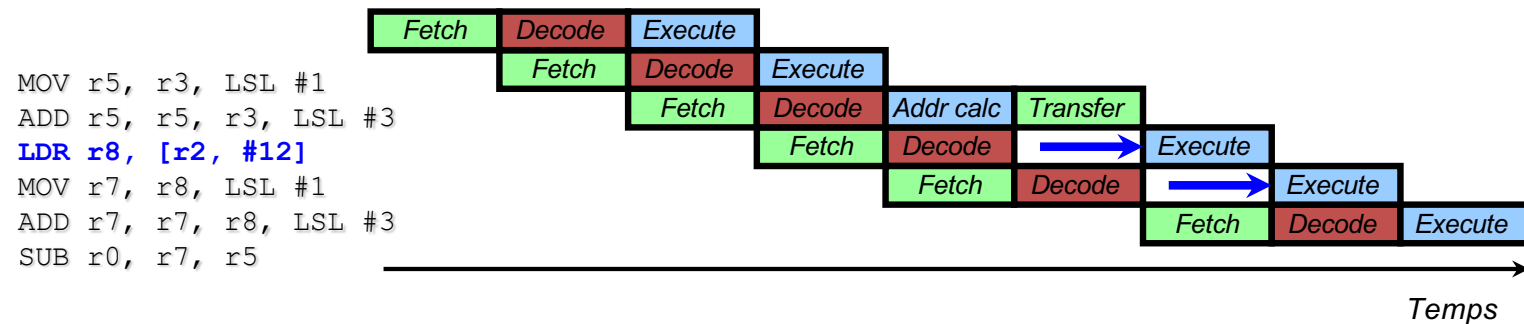
# Ralentissement du pipeline: cas des instructions d'accès mémoire (2)



Solution: décaler la dernière instruction de 1 cycle

- Décaler les phases de l'instruction SUB de 1 cycle
- MAIS: l'instruction suivante LDR (MOV) a besoin de la valeur du registre r8 qui n'est pas disponible (transfert non terminé), elle ne peut donc pas s'exécuter.
- Il faut à nouveau décaler les phases d'exécution de l'instruction MOV, et celle de l'instruction ADD

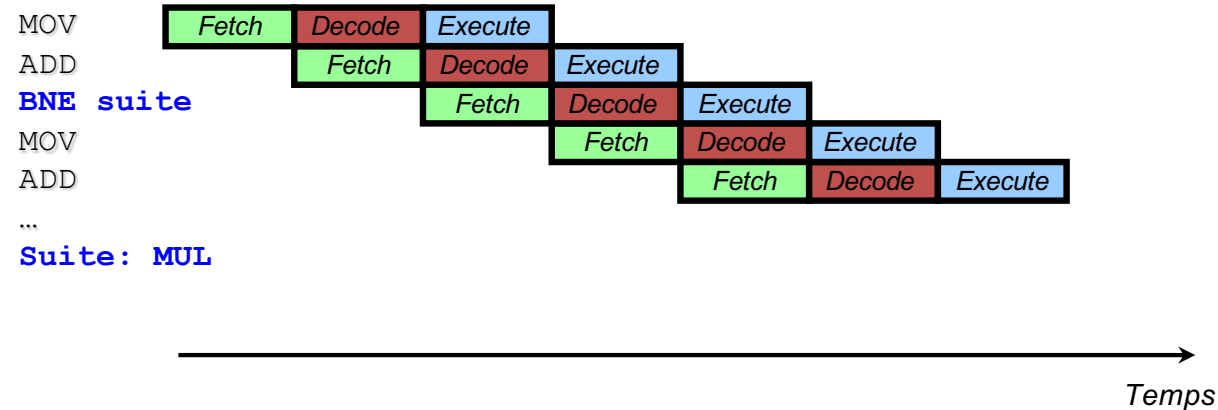
# Ralentissement du pipeline: cas des instructions d'accès mémoire (3)



Solution: décaler les phases d'exécution des 2 avant dernières instructions

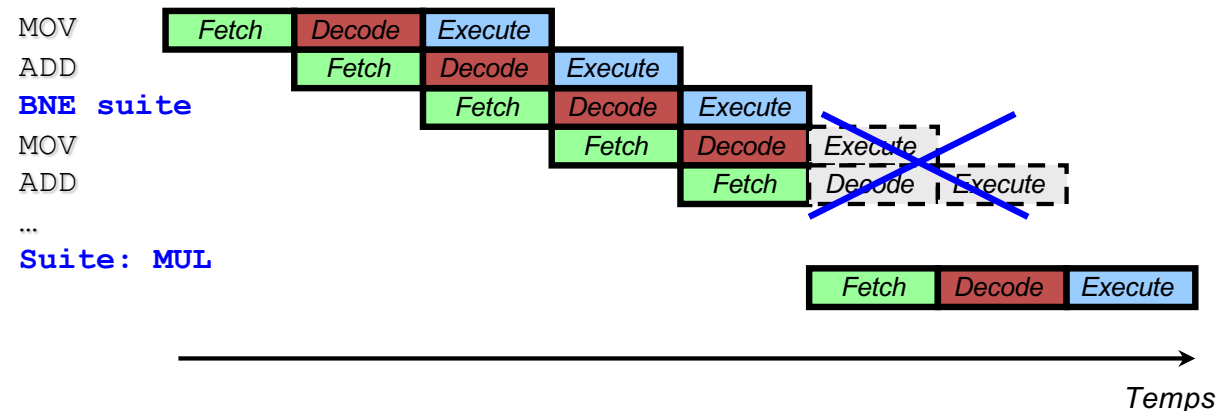
- Ralentissement du pipeline: le cycle de l'instruction en cours est arrêté jusqu'à ce que la donnée soit disponible (géré par le processeur)
- Apparition d'interruptions ("bulles") dans le pipeline
- Le pipeline implique un surcoût en complexité pour sa mise en œuvre

# Ralentissement du pipeline: cas des instructions de branchement (1)



- Si la condition de branchement (BNE) est fausse, on poursuit la séquence d'instruction (MOV, ADD). Il n'y a pas d'interruption du pipeline.

# Ralentissement du pipeline: cas des instructions de branchement (2)



- Si la condition de branchement (BNE) est vraie, on annule l'exécution des deux instructions suivantes, on poursuit à l'étiquette `Suite` (réinitialisation du pipeline).
- C'est la tâche du programmeur d'écrire du code assembleur qui utilise efficacement le pipeline pour optimiser l'exécution d'un programme
- Par exemple en utilisant l'exécution conditionnelle au lieu d'instructions de branchement (quand c'est possible), ou en ré-agénçant les instructions pour éviter les ralentissements dus aux données non disponibles.